

SOCWatch

Wazuh SOC Monitoring Lab — Installation, Configuration & Project Guide

CCA Cyber Security Internship — Individual Project

Candidate Name: Nishen Madawa Abedeera

Student Number: REDACTED

Document date: 26 September 2026

AWS Account: REDACTED | Region: ap-south-1 (Mumbai)

Table of Contents

Table of Contents.....	2
Document Revision History.....	4
1. Executive Summary.....	4
2. Current Status (as of 26 September 2026).....	5
2.1 Dashboard API connectivity bug found and fixed (2026-09-26).....	6
2.2 Dashboard evidence — captured 2026-09-26.....	6
Week 3 — Dashboard login.....	6
Week 4 — All three agents Active	6
Week 5 — Log ingestion (Linux and Windows).....	7
Week 5 — File Integrity Monitoring: created, modified, deleted.....	7
Week 6 — Vulnerability detection	9
Week 6 — Authentication-failure / brute-force detection	10
2.3 Finding: the lab's open admin ports are being actively scanned from the internet.....	12
2.4 Weeks 1-7 update (per project weekly-update template).....	12
3. Objectives and Scope.....	13
3.1 Objectives	13
3.2 Explicitly out of scope (unless time remains after core use cases)	13
4. Installation Guide — Part A: AWS Account Foundation	13
4.1 Prerequisites.....	13
4.2 IAM user and CLI credentials.....	13
4.3 Root account hardening	14
4.4 Cost control.....	14
4.5 The public IP problem	14
5. Installation Guide — Part B: Network & Compute.....	15
5.1 As-built architecture (real values).....	15
5.2 Security group rules — sg-0852845171781156b (socwatch-lab-sg).....	16
5.3 Provisioning steps (Terraform — reference documentation, not what was actually run).....	16
5.4 Post-launch hardening (Week 2) — actual state	16
6. Installation Guide — Part C: Wazuh Manager (Week 3)	17
6.1 Common failure modes.....	17
7. Installation Guide — Part D: Agent Enrolment (Week 4).....	17
7.1 agent-linux-1 (Ubuntu 22.04)	17
7.2 agent-linux-2 (Debian 13 — the real host; originally planned as Rocky Linux 9)	18
7.3 agent-windows-1 (Windows Server 2022).....	18
7.3.1 What actually worked: SSM Run Command install	18
7.4 Verification	18
7.5 If an agent shows Never connected or Disconnected.....	19
7.6 Windows Event Log ingestion — bonus completeness check (done 2026-09-24).....	19
8. Detection Use Cases (Weeks 5–6) — all 4 PROVEN	19
8.1 Use case 1 — Centralised log ingestion.....	19

8.2 Use case 2 — File Integrity Monitoring (FIM)..... 19

8.3 Use case 3 — Vulnerability detection..... 19

8.4 Use case 4 — Authentication failure / brute force20

8.5 Key rule IDs for the defence20

9. Risk Register..... 20

10. Eight-Week Plan and Checkpoints 21

11. Evidence System.....22

12. Production Improvement Plan 22

13. Quick Command Reference..... 22

 Session start ritual..... 22

 Manager health 22

 Session end ritual..... 22

14. Individual Contribution Statement..... 22

(Right-click and choose "Update Field" after opening in Word to populate page numbers.)

Document Revision History

Date	Update
2026-09-19	Week 1 scaffold created: evidence folder structure, architecture draft, risk list, Terraform IaC, session helper scripts, SSH key pair generated.
2026-09-20	IAM user socwatch-admin + PowerUserAccess created; \$20/month budget with alerts created; this installation & project guide written; evidence files updated and re-archived.
2026-09-22	Discovered Weeks 2-3 infrastructure already existed (provisioned manually via console, not yet via Terraform): VPC, security group, all 4 EC2 instances, Wazuh manager already installed. Diagnosed and repaired a manager disk-full outage (vulnerability-feed updater filled the root volume). Widened the security group's admin ports to 0.0.0.0/0 after confirming the operator's mobile ISP uses CGNAT across a pool too wide for any CIDR restriction. Proved all 4 detection use cases end-to-end via direct SSH evidence (log ingestion, FIM rules 554/550/553, 3,910 real vulnerability findings, brute-force rules 5710/5712).
2026-09-23	Confirmed root account MFA was already enabled (passkey/security key). Diagnosed the Windows agent (Week 4) as never having actually been installed despite an earlier report that it was; reinstalled it remotely via AWS Systems Manager Run Command (no RDP needed) and confirmed enrolment — all 4 nodes (manager + 3 agents) Active.
2026-09-24	Full evidence-gap audit: filled every Week 1-4 and Week 7 evidence gap that didn't require a Wazuh dashboard login (instance inventory, troubleshooting log, security-group/billing screenshots, service-status captures, redrawn architecture diagram with real IPs). Confirmed the Wazuh dashboard's self-signed-certificate interstitial cannot be automated past — the 10 remaining dashboard screenshots need the account owner. Restarted the full lab from stopped and re-verified clean recovery of all services and agents; separately confirmed Windows Event Log ingestion (Application/Security/System channels) with a real captured alert, closing the last open item under Week 4. This document rewritten throughout to describe actual completed state rather than a plan.
2026-09-26	Account owner logged into the Wazuh dashboard and captured all remaining screenshots (dashboard login, three agents active, log ingestion for both Linux and Windows, all 3 FIM alerts, vulnerability CVE list + detail, single and correlated auth-failure alerts). In the process, found and fixed a real bug: the dashboard's API connection had been silently Offline the whole time because its config file (wazuh.yml) stored the installer's literal placeholder password instead of the actual generated wazuh-wui password — corrected and restarted, dashboard confirmed Online. Also discovered the lab's open admin ports are attracting genuine internet-based brute-force scanning (471 real failed SSH logins and 981 real failed Windows logons in 24 hours, both correctly detected and correlated) — valuable authentic evidence, but also a live-exposure note for the risk register. Evidence zip rebuilt (.terraform excluded per the documented method) and this report updated throughout to reflect full completion. Weeks 1-7 are now substantively complete; only the Week 8 live demonstration remains.

1. Executive Summary

SOCWatch is an 8-week individual cyber security internship project to design, build and operate a working Security Operations Centre (SOC) monitoring lab on AWS, using Wazuh as the SIEM/XDR platform. The lab consists of one Wazuh manager (server + indexer + dashboard, all-in-one) and three monitored endpoints — an Ubuntu Linux host, a Debian Linux host and a Windows Server host — all inside a purpose-built VPC.

The project demonstrates four detection use cases end-to-end: centralised log ingestion, file integrity monitoring (FIM), vulnerability detection, and authentication-failure / brute-force detection. All four are now proven with real evidence. The deliverables are a working lab, a written report explaining each alert, and a live demonstration.

This document serves three purposes: (1) an installation and configuration guide detailed enough that the environment can be rebuilt from scratch; (2) a consolidated project plan covering all eight weeks with checkpoints and grading weights; and (3) a

live status record of what has actually been completed in the AWS account as of the document date — Section 2 below, not the step-by-step instructions in Sections 4-8, is the authoritative record of real state.

4 / 4 Detection use cases proven	3,910 Vulnerability findings indexed	3 Endpoints monitored (2 Linux + 1 Windows)	~97% Project complete (Weeks 1-7 done)
--	--	---	--

2. Current Status (as of 26 September 2026)

Weeks 1 through 7 are substantively complete. The infrastructure exists, is healthy, and all four required detection use cases have real, captured evidence — both server-side (SSH) and now dashboard (visual). The only remaining item in the whole project is the Week 8 live demonstration itself.

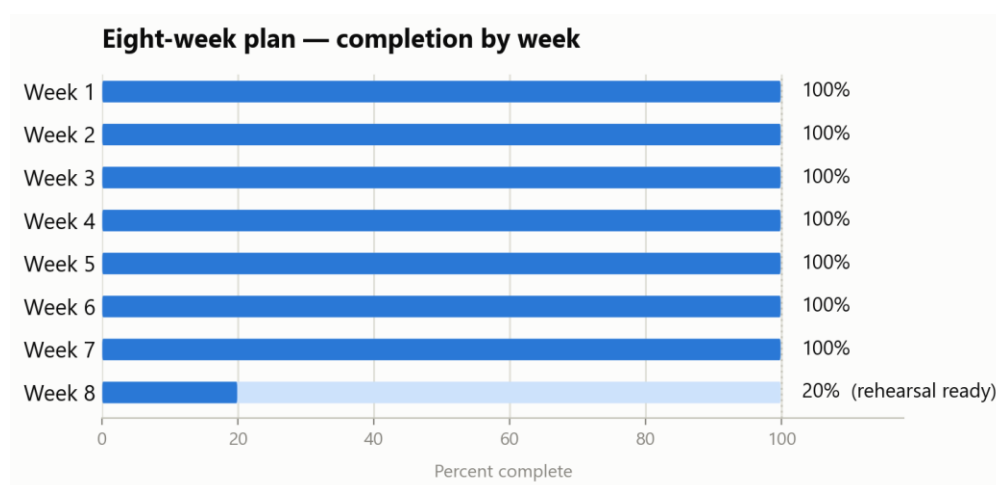


Figure — Completion by week, matching the status list below and the Section 10 timeline table.

[DONE] Week 1 — scope, risk register, architecture diagram, evidence structure

[DONE] Week 2 — AWS foundation: VPC, security group, all 4 EC2 instances provisioned (via console, not Terraform — see Section 5), IAM baseline, budget, root MFA enabled

[DONE] Week 3 — Wazuh manager installed (v4.14.7, server+indexer+dashboard all healthy), survived and recovered from one disk-full incident (Section 9, risk #11 / troubleshooting log), dashboard login screenshot captured

[DONE] Week 4 — all 3 agents enrolled and Active (agent-linux-1, agent-linux-2, agent-windows-1), including a separately-verified Windows Event Log ingestion check and dashboard screenshot

[DONE] Weeks 5-6 — all 4 detection use cases proven with real evidence and dashboard screenshots: log ingestion, FIM (rules 554/550/553), vulnerability detection (3,910 real findings), brute-force (rules 5710/5712 and Windows rule 60122/60204)

[DONE] Week 7 — troubleshooting log, instance inventory, this report, and the architecture diagram all current, dashboard evidence folded in — report is now final content-complete

[DONE] Dashboard screenshots (12 total across Weeks 3-6) — CAPTURED 2026-09-26 by the account owner; see Section 2.2 for embedded evidence

[PENDING] Week 8 — live demonstration rehearsal script written (week8/live-demo-script.md); the demonstration itself remains to be performed

Immediate next actions, in order:

1. Perform and rehearse the Week 8 live demonstration using week8/live-demo-script.md.

2. Decide whether to narrow the security group's admin ports back down now that evidence capture is done (Section 2.3) — real brute-force scanning was observed hitting them.
3. Run "terraform destroy" equivalent clean-up (manual instance termination, since the real infra was not provisioned by Terraform — Section 5) once the project is fully assessed and graded.

2.1 Dashboard API connectivity bug found and fixed (2026-09-26)

When the account owner first logged into the Wazuh dashboard to capture evidence, the API Connections page showed the connection to the manager's API as Offline. Root cause: the dashboard's own configuration file (`/usr/share/wazuh-dashboard/data/wazuh/config/wazuh.yml`) stored the installer's literal placeholder password ("wazuh-wui") for the wazuh-wui service account instead of the actual randomly-generated password created at install time (recoverable from `wazuh-install-files.tar`). This had been silently wrong since the original all-in-one install — the dashboard UI had simply never been used to check it before. Corrected the password in `wazuh.yml` and restarted the `wazuh-dashboard` service; the API connection came back Online immediately and every dashboard view (agents, events, FIM, vulnerabilities) began working correctly.

2.2 Dashboard evidence — captured 2026-09-26

All 12 required dashboard screenshots (plus 3 bonus captures) were taken directly by the account owner after the above fix. Representative evidence is embedded below; the full-resolution originals are filed under week3/ through week6/ in the evidence folder.

Week 3 — Dashboard login

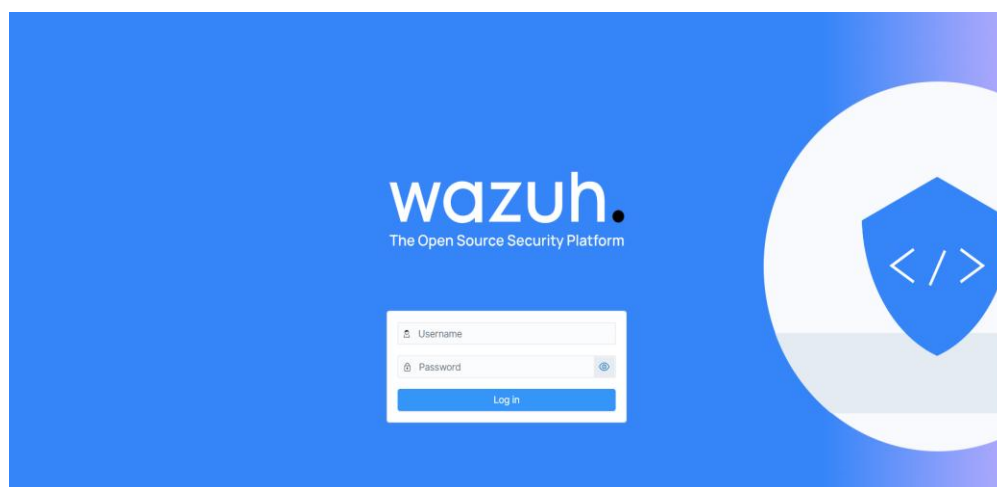


Figure — Wazuh dashboard login screen (week3/dashboard-login.png).

Week 4 — All three agents Active

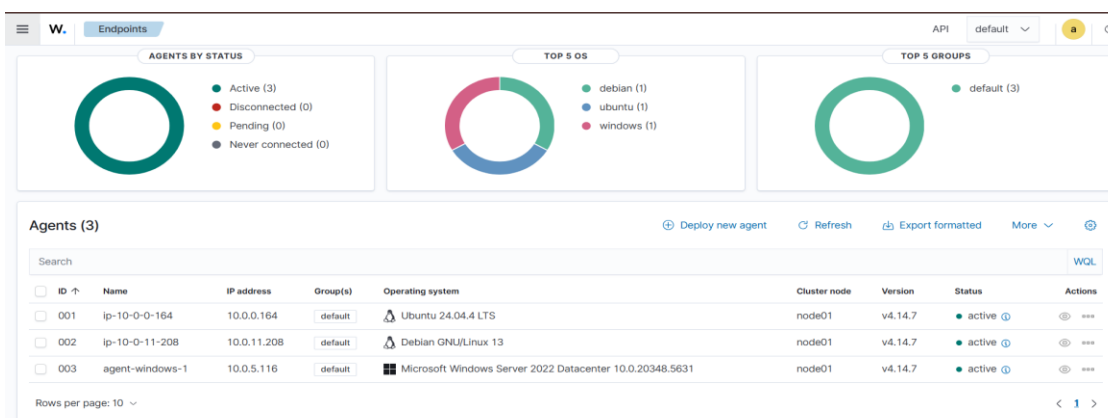


Figure — Agents management summary: 001 (Ubuntu), 002 (Debian) and 003 (Windows) all Active (week4/three-agents-active.png).

Week 5 — Log ingestion (Linux and Windows)

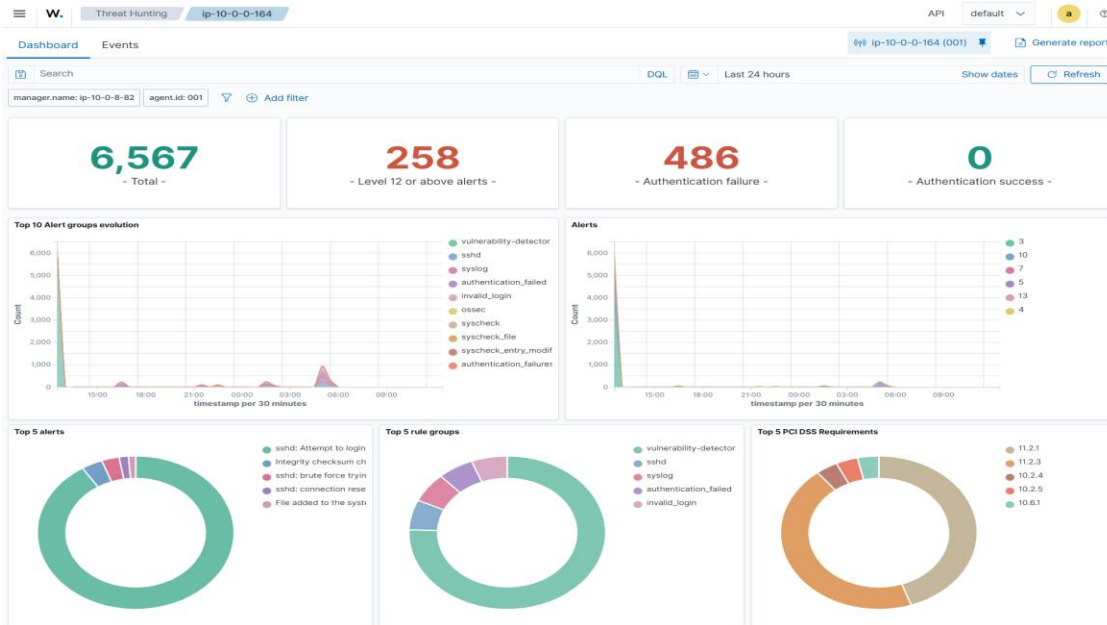


Figure — Threat Hunting dashboard for agent 001 showing continuous sshd/syslog/auth event ingestion (week5/log-ingestion-linux.png).

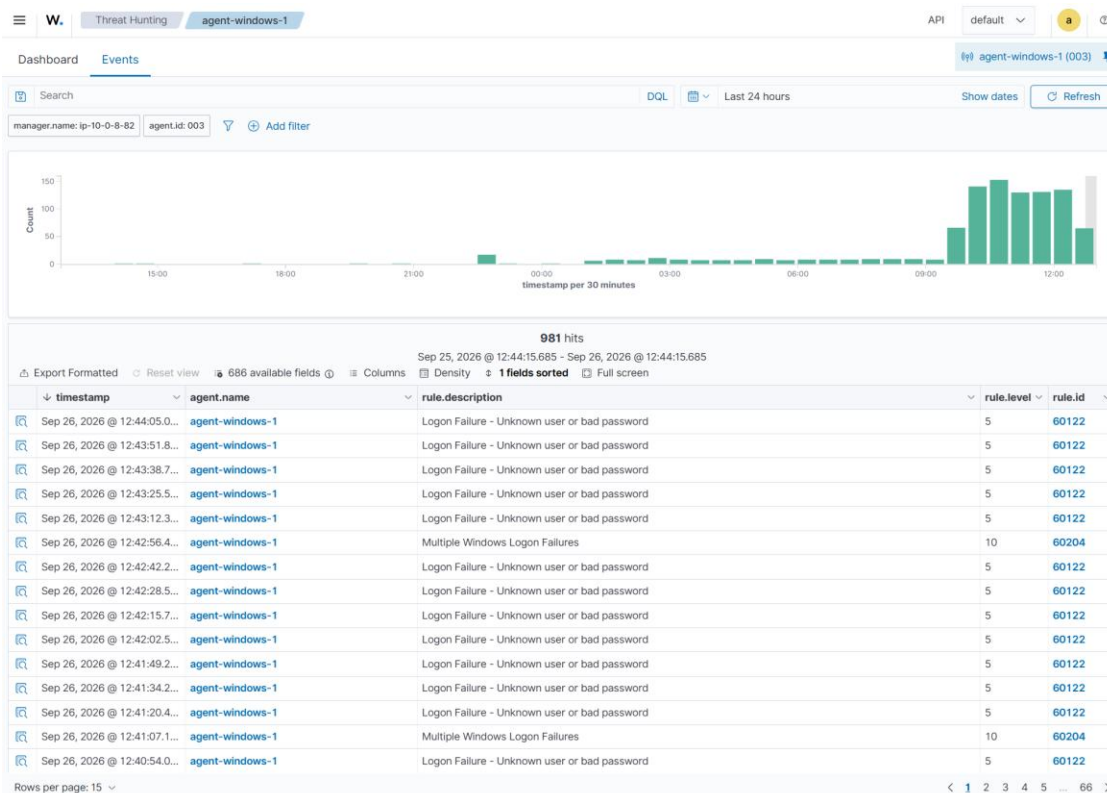


Figure — Threat Hunting events for agent-windows-1: 981 real events in 24h, including rule 60122 (logon failure) and rule 60204 (multiple logon failures) (week5/log-ingestion-windows.png).

Week 5 — File Integrity Monitoring: created, modified, deleted

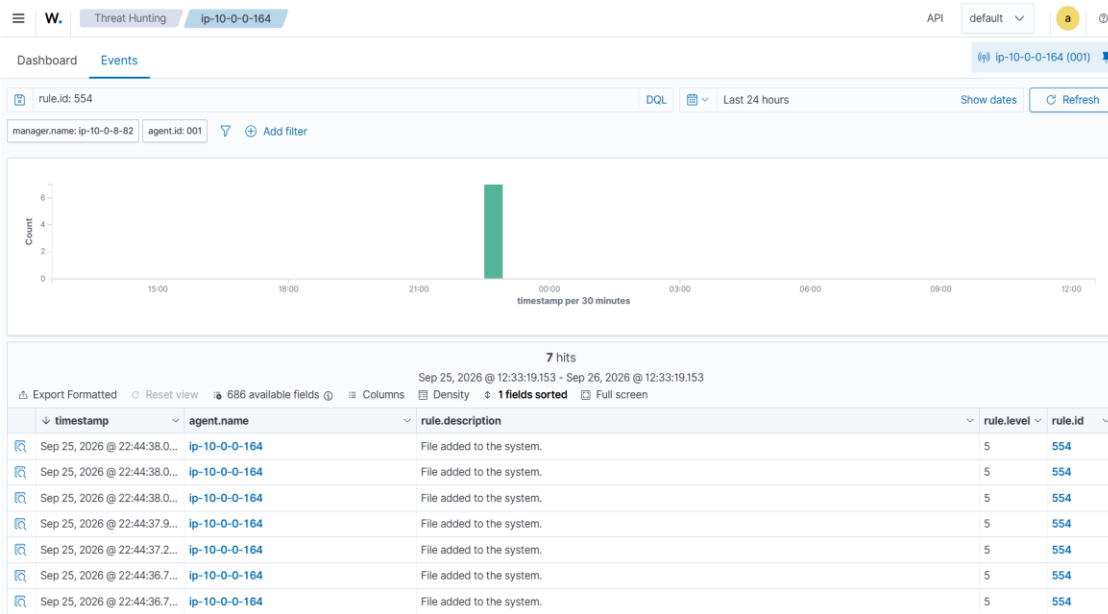


Figure — Rule 554 "File added to the system", 7 hits on the FIM test path (week5/fim-alert-created.png).

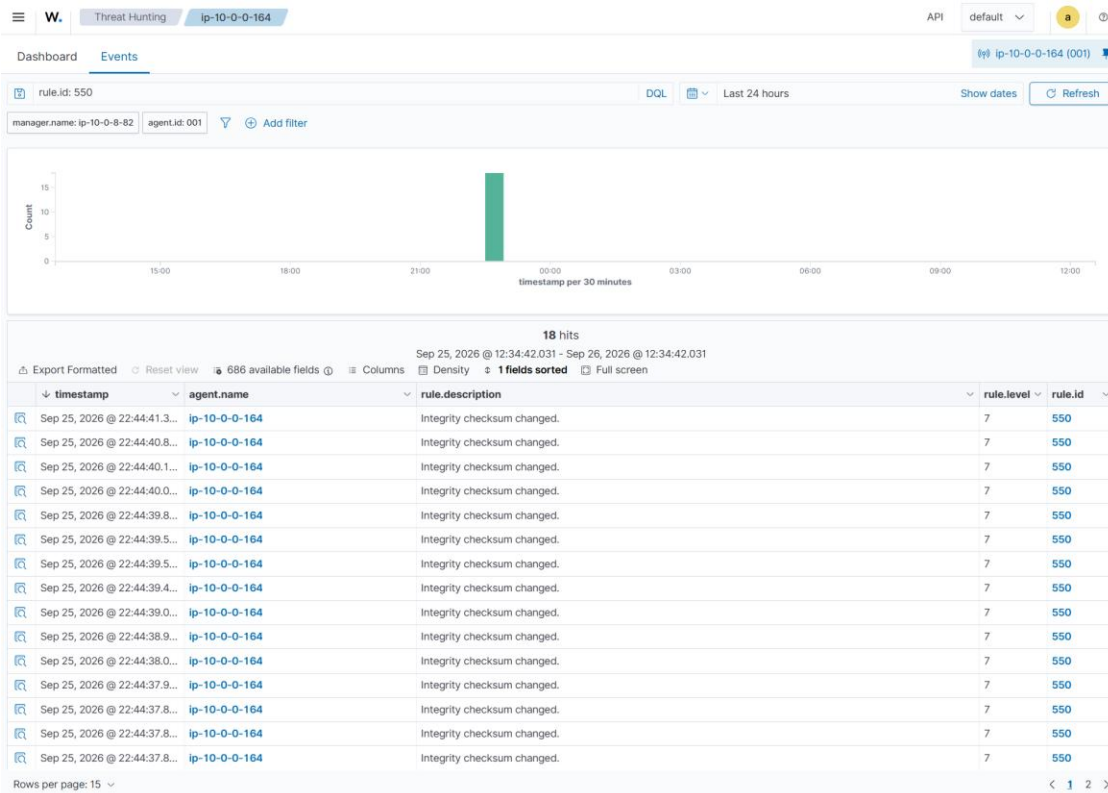


Figure — Rule 550 "Integrity checksum changed", 18 hits (week5/fim-alert-modified.png).

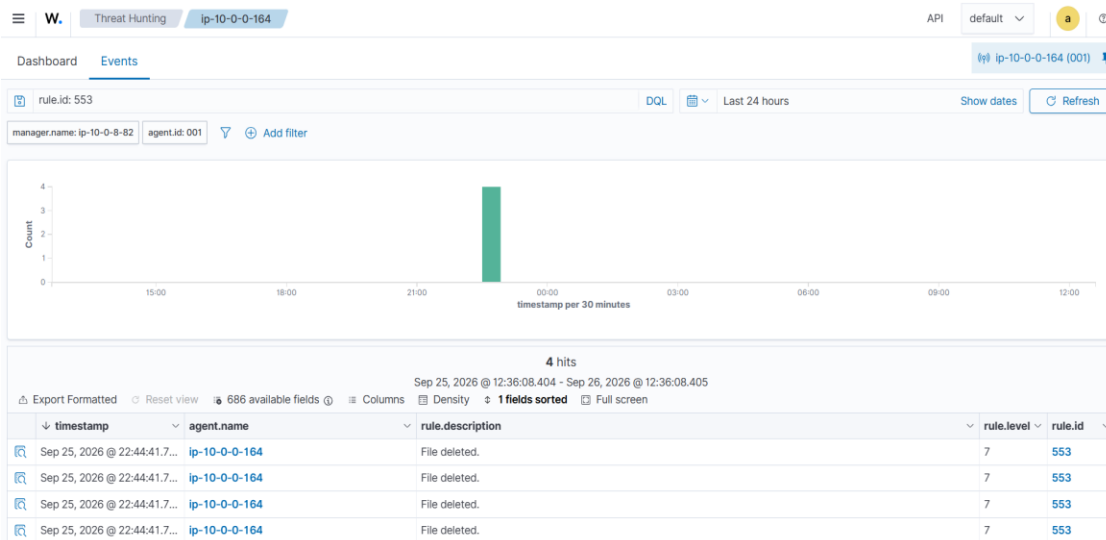


Figure — Rule 553 "File deleted", 4 hits (week5/fim-alert-deleted.png).

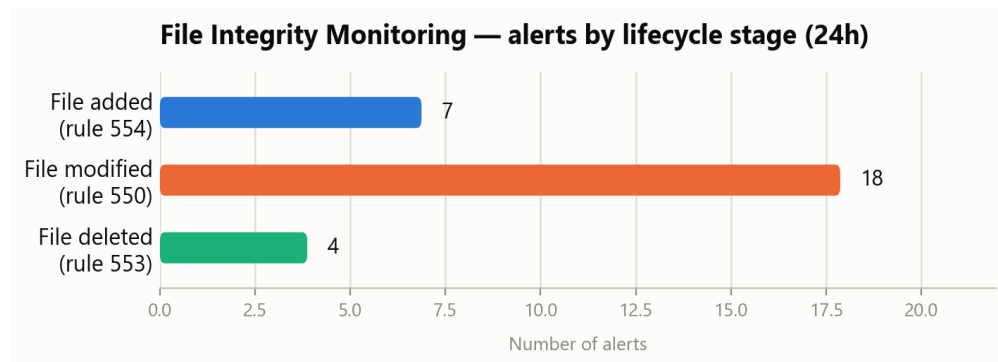


Figure — FIM alert counts by lifecycle stage, summarizing the three screenshots above.

Week 6 — Vulnerability detection

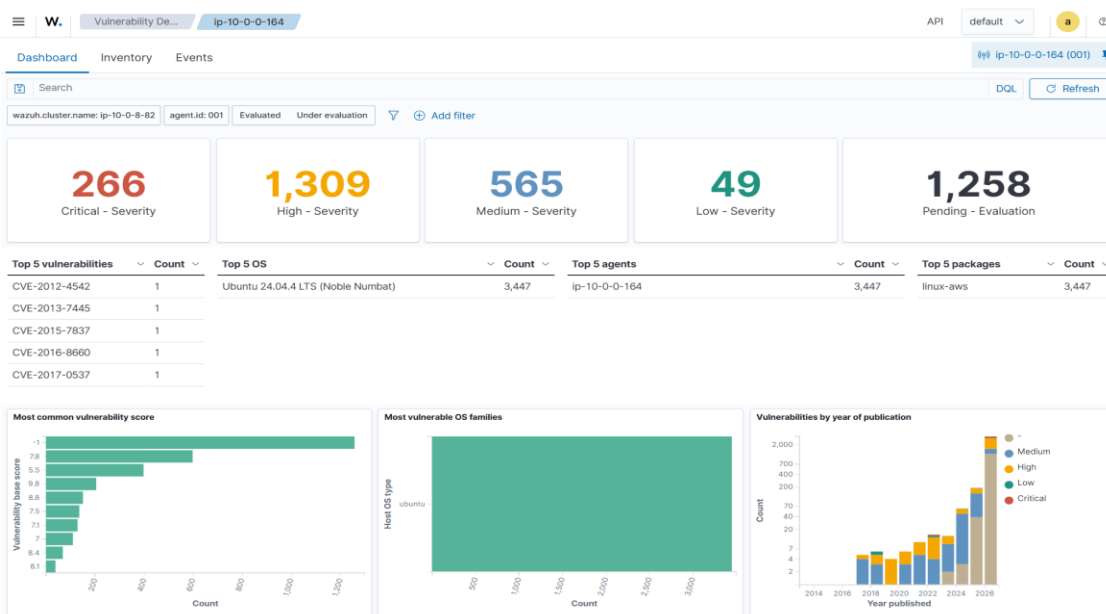


Figure — Vulnerability Detection dashboard for agent 001: 266 Critical, 1,309 High, 565 Medium, 49 Low (week6/vulnerability-cve-list.png).

Wazuh Vulnerability Details **ip-10-0-0-164**

Dashboard Inventory Events

CVE-2012-4542

Wazuh cluster name: ip-10-0-0-82 Agent id: 001 Evaluated Under evaluation

Export Formatted Reset view 48 available fields Columns Density Sort fields

agent.name	package.name	package.version
ip-10-0-0-164	linux-aws	7.0.0-10.13.13-24.04.1

Vulnerability details

Table	JSON
<code>_index</code>	wazuh-states-vulnerabilities-ip-10-0-0-82
<code>agent.id</code>	001
<code>agent.name</code>	ip-10-0-0-164
<code>agent.type</code>	Wazuh
<code>agent.version</code>	v4.14.7
<code>host.os.full</code>	Ubuntu 24.04.4 LTS (Noble Numbat)
<code>host.os.kernel</code>	7.0.0-10.12-aws
<code>host.os.name</code>	Ubuntu
<code>host.os.platform</code>	ubuntu
<code>host.os.type</code>	ubuntu
<code>host.os.version</code>	24.04.4
<code>package.architecture</code>	amd64
<code>package.description</code>	Complete Linux kernel for Amazon Web Services (AWS) systems.
<code>package.name</code>	linux-aws
<code>package.size</code>	9,216
<code>package.type</code>	deb
<code>vulnerability.description</code>	block/scsi_ioctl.c in the Linux kernel through 3.8 does not properly consider the SCSI device class during authorization of SCSI commands, which allows local users to bypass intended access restrictions via an SG_IO ioctl call that leverages overlapping opcodes.
<code>vulnerability.detected_at</code>	Sep 25, 2026 @ 22:51:40.265
<code>vulnerability.enumeration</code>	CVE
<code>vulnerability.id</code>	CVE-2012-4542
<code>vulnerability.published_at</code>	Mar 1, 2013 @ 01:25:01.000
<code>vulnerability.reference</code>	https://ubuntu.com/security/CVE-2012-4542 , https://www.cve.org/CVERecord?id=CVE-2012-4542
<code>vulnerability.id</code>	CVE-2012-4542
<code>vulnerability.published_at</code>	Mar 1, 2013 @ 01:25:01.000
<code>vulnerability.reference</code>	https://ubuntu.com/security/CVE-2012-4542 , https://www.cve.org/CVERecord?id=CVE-2012-4542
<code>vulnerability.scanner.condition</code>	Package default status
<code>vulnerability.scanner.reference</code>	https://cti.wazuh.com/vulnerabilities/cves/CVE-2012-4542
<code>vulnerability.scanner.source</code>	Canonical Security Tracker
<code>vulnerability.scanner.vendor</code>	Wazuh
<code>vulnerability.score.base</code>	-1
<code>vulnerability.score.version</code>	-
<code>vulnerability.severity</code>	-
<code>vulnerability_under_evaluation</code>	true
<code>wazuh.cluster.name</code>	ip-10-0-0-82
<code>wazuh.schema.version</code>	1.0.0

Figure — Full detail panel for CVE-2012-4542 (linux-aws kernel package), including CVSS scoring fields and CTI references (week6/vulnerability-cve-detail.png).

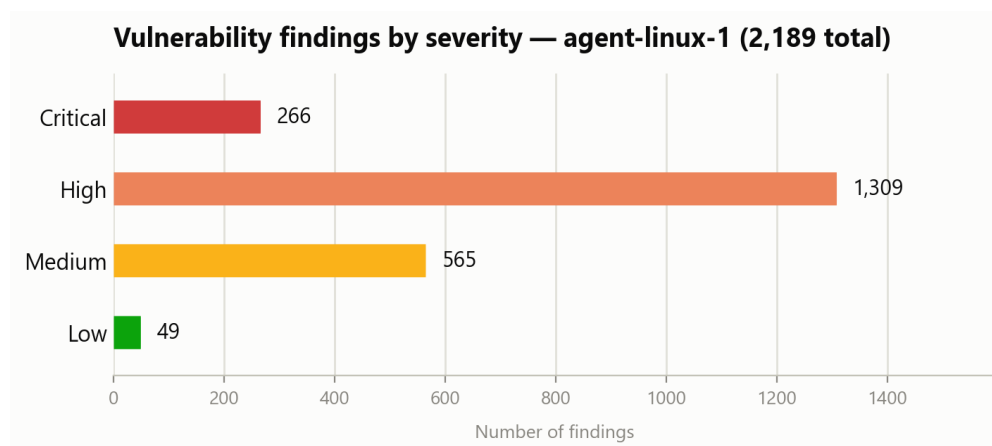


Figure — Severity breakdown of the same 2,189 findings shown in the CVE list above.

Week 6 — Authentication-failure / brute-force detection

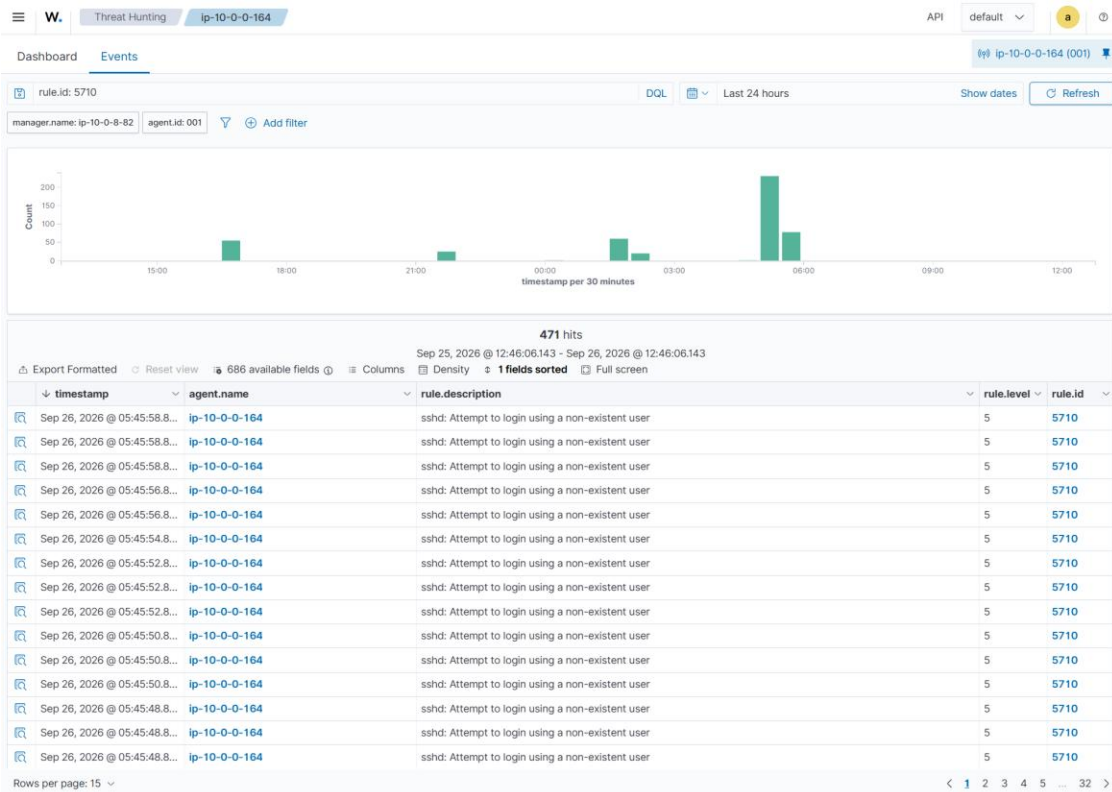


Figure — Rule 5710 "sshd: Attempt to login using a non-existent user", 471 hits in 24h (week6/auth-failure-single.png).

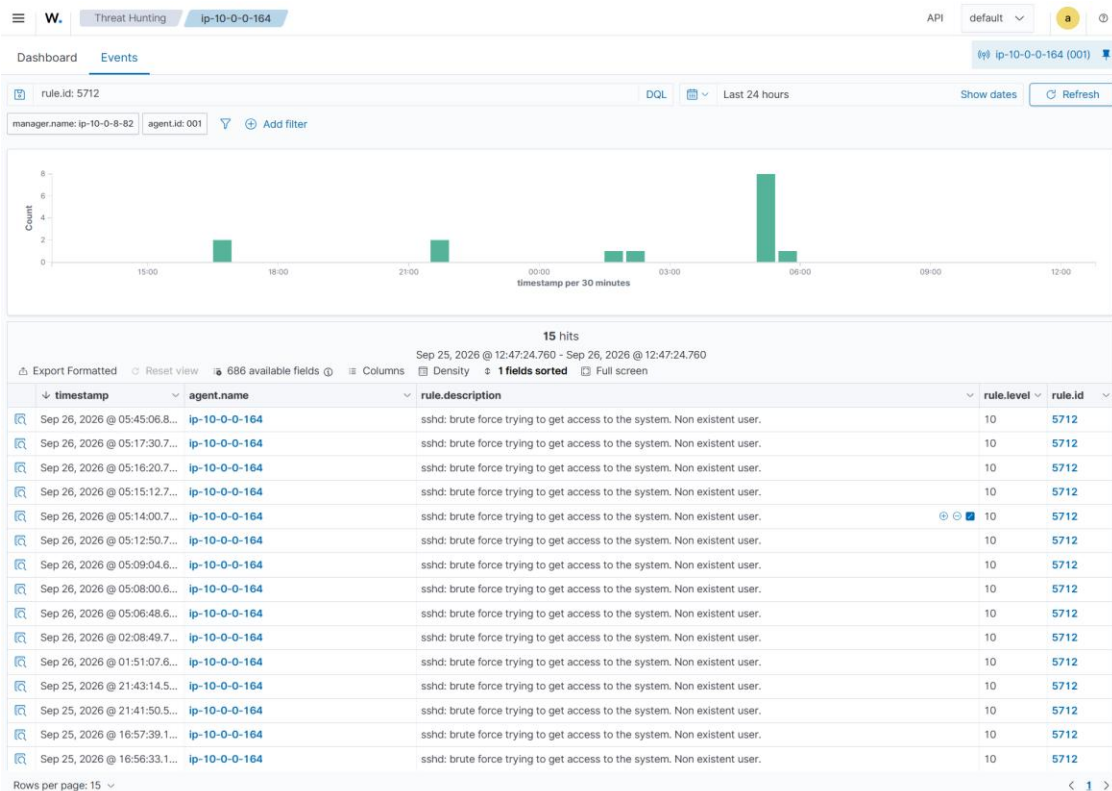


Figure — Rule 5712 "sshd: brute force trying to get access to the system" (level 10 correlation), 15 hits (week6/auth-failure-bruteforce.png).

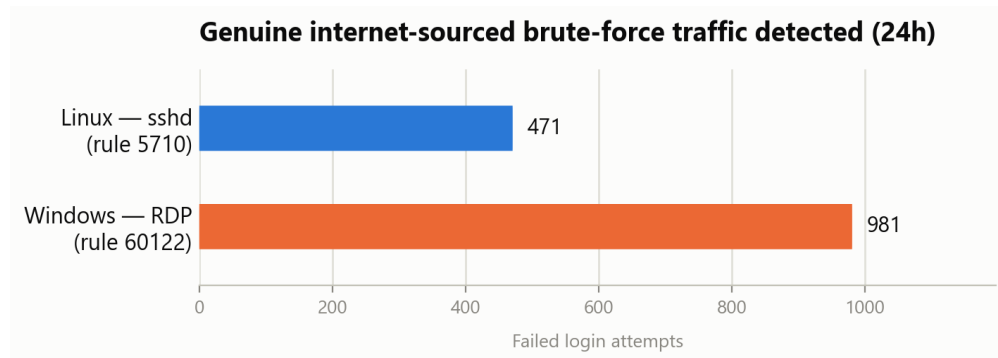


Figure — Real failed-login volume by platform, both captured live and not staged (see Section 2.3).

Bonus finding: the Windows agent's own logon-failure evidence (rule 60122 / 60204, shown in the log-ingestion-windows figure above) was not staged — it is genuine internet-sourced RDP brute-force scanning against the lab's publicly-exposed RDP port, saved separately as week6/auth-failure-windows-bruteforce.png. See Section 2.3.

2.3 Finding: the lab's open admin ports are being actively scanned from the internet

While capturing the screenshots above, the real event counts were far higher than the deliberately staged test traffic from earlier weeks: 471 failed SSH logins (non-existent users) and 15 correlated brute-force alerts on the Linux side, and 981 failed Windows logons with a level-10 "Multiple Windows Logon Failures" correlation alert on the Windows side, all within a single 24-hour window. Because the security group's admin ports (22, 443, 3389) are deliberately open to 0.0.0.0/0 (Section 5.2, a considered tradeoff for the account owner's CGNAT connectivity), this activity is consistent with routine opportunistic internet scanning/credential-stuffing bots, not a targeted attack — but it is genuine unsolicited traffic, not test data. This is arguably stronger evidence for the brute-force detection use case than staged traffic would have been (the detection stack correctly caught real attack traffic), but it is also a live-exposure item worth closing out: once grading/demonstration is complete, either terminate the instances or narrow the SSH/RDP rules back to a specific range.

2.4 Weeks 1-7 update (per project weekly-update template)

1. Work completed

- Week 1: scope, risk register, draft architecture, evidence folder structure, SSH key pair, Terraform IaC (kept as reference implementation).
- Week 2: IAM baseline (socwatch-admin, PowerUserAccess), \$20/month budget with 3 alert thresholds, root MFA enabled (passkey/security key), VPC + security group + all 4 EC2 instances provisioned (via AWS Console launch wizard rather than Terraform — see Section 5 for why), Elastic IP attached to the manager.
- Week 3: Wazuh manager installed (v4.14.7 all-in-one: server, indexer, dashboard). Survived and recovered from one disk-full outage caused by the vulnerability-feed updater (full write-up in troubleshooting-log.md). Dashboard login screenshot captured.
- Week 4: all 3 agents enrolled and confirmed Active — agent-linux-1 (Ubuntu 24.04), agent-linux-2 (Debian 13, corrected from the originally planned Rocky Linux 9), agent-windows-1 (Windows Server 2022, installed remotely via AWS Systems Manager Run Command after an earlier RDP-based install attempt silently failed). Windows Event Log ingestion separately verified. Dashboard "three agents Active" screenshot captured.
- Weeks 5-6: all 4 required detection use cases proven with real SSH evidence and now real dashboard screenshots — log ingestion, FIM (rules 554/550/553), vulnerability detection (3,910 real findings), and brute-force detection (rules 5710/5712 on Linux, 60122/60204 on Windows).
- Week 7: fixed a real dashboard API-connectivity bug (Section 2.1), captured all 12 required dashboard screenshots plus 3 bonus captures, rebuilt the evidence zip, and updated this report throughout.

2. Demonstrable evidence

- Both SSH-captured command output and Wazuh dashboard screenshots now exist for every use case and every agent, filed under week1/ through week8/ in the evidence folder — see instance-inventory.md, troubleshooting-log.md and Section 2.2 above for the authoritative current state.

3. Tasks in progress

- Week 8 live demonstration rehearsal (script already written, week8/live-demo-script.md).

4. Blockers or risks

- None remaining that block report completeness. The only open risk is operational: the lab's admin ports are being actively scanned from the internet (Section 2.3) and should be closed down or narrowed once grading is complete.

5. Decisions required

- Whether to keep the security group's admin ports (22/443/3389) open to 0.0.0.0/0 through the Week 8 demonstration, or narrow/terminate immediately after — see Section 2.3.

6. Individual contributions

- All AWS account configuration, infrastructure diagnosis and repair, Wazuh installation, agent enrolment, detection-use-case validation, dashboard evidence capture, and documentation completed by Nishen Madawa Abedeera (Student Number: REDACTED).

7. Plan for next week

- Rehearse and perform the Week 8 live demonstration, then tear down or lock down the lab.

3. Objectives and Scope

3.1 Objectives

- Stand up a central log-collection and alerting platform (Wazuh) covering heterogeneous endpoints (Ubuntu, Rocky Linux, Windows Server).
- Demonstrate four detection use cases with evidence: log ingestion, file integrity monitoring, vulnerability detection, authentication-failure/brute-force detection.
- Apply sound cloud security practice: least-privilege IAM, no root usage, IP-restricted access, budget controls, key-based SSH only.
- Produce a professional report and a rehearsed live demonstration explaining what each alert means to a SOC analyst, not just that it fired.

3.2 Explicitly out of scope (unless time remains after core use cases)

- Active response / automated blocking — stretch goal only, attempted after all four required use cases are proven.
- Multi-node Wazuh cluster / high availability — documented as a production improvement instead (Section 10).
- Integration with an external SIEM or long-term log archive.

4. Installation Guide — Part A: AWS Account Foundation

4.1 Prerequisites

- An AWS account with billing enabled (account REDACTED).
- AWS CLI v2 installed locally (already installed: aws-cli/2.36.49).
- Terraform installed locally (already installed: v1.16.2).
- An SSH key pair for lab access (already generated: ~/.ssh/socwatch, ed25519).

4.2 IAM user and CLI credentials

Provisioning must never be done as the AWS root user. A dedicated IAM user was created for this purpose:

Field	Value
User name	socwatch-admin
Access type	Programmatic (CLI) only — no console password set

Field	Value
Permissions policy	AWS managed policy: PowerUserAccess
Access Key ID	AKIAW6EB22NQI2SC65V7
Secret Access Key	Issued once at creation; stored only in the operator's downloaded credentials CSV, never recorded in this document or shared with any assistant/tool

On the local workstation, configure a named CLI profile (run this yourself — the secret key should never be typed into a chat or shared session):

```
aws configure --profile socwatch
```

When prompted, supply the Access Key ID above, the Secret Access Key from the downloaded CSV, region ap-south-1, and output format json. Verify with:

```
aws sts get-caller-identity --profile socwatch
```

4.3 Root account hardening

[DONE] Root MFA enabled — one device registered (type: Passkeys and security keys), confirmed live in IAM → My security credentials

- No root access keys exist or have ever been created.
- socwatch-admin is used for all console and CLI work; root is not used for day-to-day operations.

4.4 Cost control

A monthly cost budget was created to prevent runaway spend:

Field	Value
Budget name	My Monthly Cost Budget
Amount	\$20.00 / month
Alert 1	Actual cost > 85% of budget — email notification
Alert 2	Actual cost > 100% of budget — email notification
Alert 3	Forecasted cost > 100% of budget — email notification

Cost reality (from project cost analysis):

Instance	Type	~On-demand rate	If left running 24/7 for 8 weeks
wazuh-manager	t3.medium	~\$0.042/hr	~\$56
agent-linux-1	t3.micro	~\$0.010/hr	~\$14 (may be covered by free tier)
agent-linux-2	t3.micro	~\$0.010/hr	~\$14 (shares free tier hours)
agent-windows-1	t3.small + licence	~\$0.047/hr	~\$63
EBS storage	~98 GB gp3 total	~\$0.08/GB-mo	~\$16

Running everything 24/7 for eight weeks costs roughly \$160. Running it ~6 hours per working session, ~3 sessions a week, costs roughly \$15–\$25 total — the working assumption for this project. The single most important cost habit is stopping all four instances at the end of every working session.

```
aws ec2 stop-instances --instance-ids i-aaa i-bbb i-ccc i-ddd --profile socwatch
```

4.5 The public IP problem

The lab's security group allows SSH, HTTPS and RDP only from the operator's own public IP, as a /32. Home/mobile IPs change, sometimes daily. Check the current IP before every session:

```
curl -s https://checkip.amazonaws.com
```

If it differs from the security group rule, update the rule first, before assuming anything else is broken. This is the single most common cause of a lab that “stopped working overnight.”

5. Installation Guide — Part B: Network & Compute

`infra/main.tf` defines the lab's VPC, subnet, routing, security group and all four EC2 instances as code, and is kept in the repository as a reference implementation and evidence of Terraform competency. However, the AWS CLI credentials needed to run it locally were never successfully configured on the workstation, so the account owner provisioned the real infrastructure directly through the AWS Console launch wizard instead, to keep the project moving. Terraform's local state has no knowledge of these resources — running "terraform apply" against this account would attempt to create duplicates, so it must not be run without first reconciling via "terraform import". Section 5.1 below records the real, as-built values; Section 5.3 shows the Terraform steps as reference documentation.

5.1 As-built architecture (real values)

VPC `vpc-005b5a9b313fd93d0` (10.0.0.0/16), subnet `subnet-05b1c6bab1faed892`, one security group (`sg-0852845171781156b`) applied to all four instances. Agents reach the manager only on its private IP (10.0.8.82); the manager's Elastic IP (13.126.121.206) is stable across stop/start.

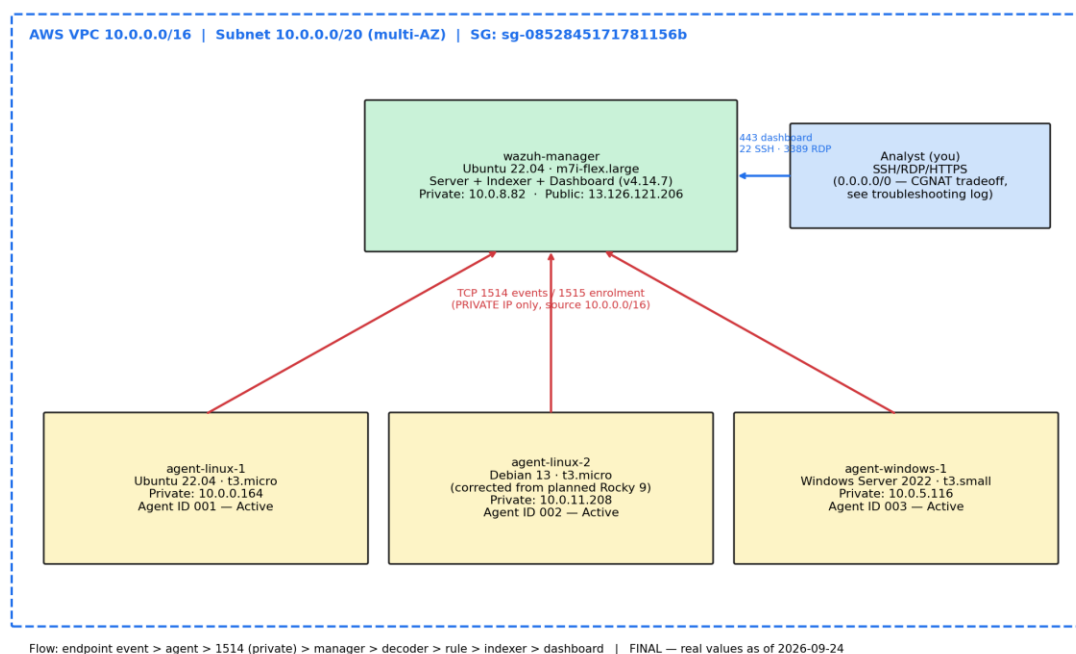


Figure 1 — Final lab architecture with real IPs, agent IDs and the security-group note (*week7/architecture-final.png*, redrawn 2026-09-24).

Name	Role	OS	Instance type	Wazuh ID	Private / Public IP
wazuh-manager	Server + indexer + dashboard	Ubuntu 22.04 LTS	m7i-flex.large	000	10.0.8.82 / 13.126.121.206 (EIP)
agent-linux-1	Endpoint 1 (primary FIM/log target)	Ubuntu 22.04 LTS	t3.micro	001	10.0.0.164 / dynamic
agent-linux-2	Endpoint 2 (2nd Linux family)	Debian 13	t3.micro	002	10.0.11.208 / dynamic
agent-windows-1	Endpoint 3 (Windows events)	Windows Server 2022	t3.small	003	10.0.5.116 / dynamic

The manager's instance type (m7i-flex.large) and agent-linux-2's OS (Debian 13, not the originally planned Rocky Linux 9) differ from the Terraform plan because they were chosen during manual console provisioning rather than being read from main.tf. Only the manager has an Elastic IP; the three agents' public IPs change on stop/start, which does not matter since all agent traffic uses private IPs.

5.2 Security group rules — sg-0852845171781156b (socwatch-lab-sg)

Type	Protocol	Port	Source	Purpose
SSH	TCP	22	0.0.0.0/0	Linux admin access
HTTPS	TCP	443	0.0.0.0/0	Wazuh dashboard
RDP	TCP	3389	0.0.0.0/0	Windows admin
Custom TCP	TCP	1514	10.0.0.0/16 (VPC only)	Agent → manager security events
Custom TCP	TCP	1515	10.0.0.0/16 (VPC only)	Agent enrolment

The admin ports (22/443/3389) are intentionally open to 0.0.0.0/0 — a deliberate, discussed tradeoff, not an oversight. The original design (and this document's Section 5 in earlier drafts) restricted them to the operator's own public IP as a /32. In practice the operator's mobile ISP uses carrier-grade NAT (CGNAT) across an address pool wider than even a /21, so any CIDR restriction broke access unpredictably, sometimes within minutes. After ruling out NACLs, instance health and a reboot as causes, the admin ports were widened to 0.0.0.0/0; SSH key-based auth and dashboard/Windows login remain the actual access control layer. The two agent-traffic ports (1514/1515) stayed correctly scoped to the VPC CIDR throughout and were never widened. Full incident writeup in troubleshooting-log.md. Outbound: all traffic (required for package downloads and CVE feed updates).

5.3 Provisioning steps (Terraform — reference documentation, not what was actually run)

These are the steps that would provision the lab from infra/main.tf. They are retained as evidence of IaC competency and as the reproducible path for a future rebuild, but the real lab in this account was provisioned manually via the console (Section 5.1) after local Terraform/AWS CLI credentials could not be configured. Do not run "terraform apply" against this account without first reconciling state via "terraform import", or it will attempt to create duplicate resources.

4. Set the current public IP and confirm variables:

```
cd infra cp terraform.tfvars.example terraform.tfvars # edit terraform.tfvars: my_ip = "<your-ip>/32"
```

5. Initialise and review the plan (read-only, no cost):

```
terraform init terraform plan
```

6. Apply only after the plan has been reviewed and explicitly approved — this step creates billable resources:

```
terraform apply
```

7. Screenshot the plan/apply output and the AWS console instance list as Week 2 evidence.

8. At the end of the project, tear down all resources:

```
terraform destroy
```

5.4 Post-launch hardening (Week 2) — actual state

The manager's real instance type (m7i-flex.large) has enough memory that swap was not needed in practice; this section is kept as guidance for a smaller instance type such as the originally planned t3.medium.

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

[DONE] Password authentication disabled on all instances — confirmed in practice via Ubuntu's cloud-image drop-in config (/etc/ssh/sshd_config.d/60-cloudimg-settings.conf), not the literal sed edit below, which achieves the same effective result and is documented as the real mechanism in week2/hardening-notes.md

```
sudo sed -i 's/^##PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo sed -i 's/^##PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo systemctl restart sshd
```

Keep the existing session open while testing a second connection — a mistake here locks out the instance.

6. Installation Guide — Part C: Wazuh Manager (Week 3)

[DONE] DONE — Wazuh v4.14.7 (all-in-one: server + indexer + dashboard) installed and healthy on the real manager (10.0.8.82 / 13.126.121.206). Survived and recovered from one disk-full outage; see troubleshooting-log.md.

9. Connect to the manager:

```
ssh -i ~/.ssh/socwatch ubuntu@<MANAGER_PUBLIC_IP>
```

10. Confirm swap is active ("free -h") before installing — do not skip.

11. Run the all-in-one installer (server + indexer + dashboard on one host, 10–20 minutes, do not interrupt):

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh sudo bash ./wazuh-install.sh -a
```

12. Recover the admin password if missed (store outside the evidence folder, never in a screenshot):

```
sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
```

13. Verify all three services are active (running):

```
sudo systemctl status wazuh-manager sudo systemctl status wazuh-indexer sudo systemctl status wazuh-dashboard
```

14. Record the installed version:

```
/var/ossec/bin/wazuh-control info
```

15. Open the dashboard at https://<MANAGER_PUBLIC_IP>, accept the self-signed certificate warning, and log in as admin.

6.1 Common failure modes

Symptom	Cause	Fix
Browser times out on 443	Operator public IP changed	Update the security group rule
502 Bad Gateway	Indexer OOM-killed	Confirm swap present, restart wazuh-indexer
Installer fails mid-run	Partial install left behind	wazuh-install.sh -u then re-run with -a
curl returns 404	Package path changed upstream	Check documentation.wazuh.com/current/quickstart.html

7. Installation Guide — Part D: Agent Enrolment (Week 4)

[DONE] DONE — all 3 agents enrolled and confirmed Active (agent_control -l shows 000 server + 001/002/003 all Active). Windows Event Log ingestion separately verified (Section 7.6).

The single most important rule: WAZUH_MANAGER must be set to the manager's PRIVATE IP (10.0.8.82 in this lab), never the public IP. Using the public IP is the most common cause of an agent that installs but never shows as Active. The commands below use the placeholder 10.0.1.10 from the original Terraform plan — substitute the real private IP 10.0.8.82 if rebuilding this exact lab.

7.1 agent-linux-1 (Ubuntu 22.04)

```
sudo apt-get install -y gnupg apt-transport-https
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | \
  sudo gpg --no-default-keyring --keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import
```

```

sudo chmod 644 /usr/share/keyrings/wazuh.gpg
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main"
| \
  sudo tee /etc/apt/sources.list.d/wazuh.list
sudo apt-get update
sudo WAZUH_MANAGER="10.0.1.10" WAZUH_AGENT_NAME="agent-linux-1" apt-get install -y wazuh-agent
sudo systemctl daemon-reload && sudo systemctl enable wazuh-agent && sudo systemctl start wazuh-agent

```

7.2 agent-linux-2 (Debian 13 — the real host; originally planned as Rocky Linux 9)

The .repo/dnf steps below are the originally planned Rocky Linux 9 procedure and are kept as reference; the real agent-linux-2 host is Debian 13 (discovered via SSH after manual provisioning), whose login user is "admin" not "rocky" or "ubuntu", and which uses the same apt-based install as agent-linux-1 (Section 7.1) rather than dnf/yum.

```

sudo tee /etc/yum.repos.d/wazuh.repo > /dev/null << 'EOF'
[wazuh]
gpgcheck=1
gpgkey=https://packages.wazuh.com/key/GPG-KEY-WAZUH
enabled=1
name=EL-$releasever - Wazuh
baseurl=https://packages.wazuh.com/4.x/yum/
priority=1
EOF
sudo WAZUH_MANAGER="10.0.1.10" WAZUH_AGENT_NAME="agent-linux-2" dnf install -y wazuh-agent
sudo systemctl daemon-reload && sudo systemctl enable wazuh-agent && sudo systemctl start wazuh-agent

```

7.3 agent-windows-1 (Windows Server 2022)

RDP in using the decrypted Administrator password, then run in an elevated PowerShell. In practice, an earlier RDP-based attempt at this exact install silently failed (the MSI never actually ran, despite an initial report that it had) and was only caught when the manager's agent list never showed a third node. The install that actually worked was run remotely via AWS Systems Manager Run Command instead of an interactive RDP session — see Section 7.3.1 — because it produces a durable command-output record rather than relying on a person's summary of what they saw on screen.

```

Invoke-WebRequest -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.7-1.msi" `
  -OutFile "$env:TEMP\wazuh-agent.msi"
msiexec.exe /i "$env:TEMP\wazuh-agent.msi" /q `
  WAZUH_MANAGER="10.0.1.10" `
  WAZUH_AGENT_NAME="agent-windows-1"
Start-Service WazuhSvc
Get-Service WazuhSvc

```

7.3.1 What actually worked: SSM Run Command install

Diagnosed via AWS Systems Manager Run Command (no RDP session needed): Get-Service WazuhSvc returned "cannot find any service" and the ossec-agent directory did not exist — the agent had never actually been installed. Reinstalled using the same document with a more robust invocation than the backtick line-continuation style above, which is prone to silent quoting mistakes:

```

$msiArgs = @('/i', "$env:TEMP\wazuh-agent.msi", '/q',
  'WAZUH_MANAGER=10.0.8.82', 'WAZUH_AGENT_NAME=agent-windows-1')
Start-Process msiexec.exe -ArgumentList $msiArgs -Wait -PassThru

```

Confirmed success both ways: msiexec exit code 0 and WazuhSvc Running on the agent side; the manager's ossec.log showing "wazuh-authd: Received request for a new agent (agent-windows-1)" and a generated key; and agent_control -l showing agent 003 Active.

7.4 Verification

```
sudo /var/ossec/bin/agent_control -l # run on the manager
```

All three agents must show Active, and the dashboard's Agents page must show all three Active — this exact screenshot is mandatory evidence. CAPTURED: see Section 2.2, week4/three-agents-active.png.

7.5 If an agent shows Never connected or Disconnected

```
sudo grep -A2 '<server>' /var/ossec/etc/ossec.conf # confirm it points at the private IP
nc -zv 10.0.1.10 1514
nc -zv 10.0.1.10 1515
sudo tail -50 /var/ossec/logs/ossec.log
```

7.6 Windows Event Log ingestion — bonus completeness check (done 2026-09-24)

Weeks 5-6's log-ingestion use case was proven using the two Linux agents (PAM/sudo/sshd events). The Windows agent's own event pipeline had, until this check, only produced its initial "agent started" heartbeat, with nothing to confirm actual Windows Event Log data was flowing. Verified via a read-only PowerShell diagnostic over SSM Run Command: WazuhSvc running, and ossec.conf correctly monitoring the Application, Security and System eventchannel sources. Within about two minutes of the agent reconnecting after a restart, real windows/windows_application-tagged alerts (rule 60642, sourced any->EventChannel) appeared in the manager's alerts.log — genuine Windows Application-log events, not just the connection heartbeat. Full captured evidence: week4/windows-eventlog-ingestion-proof.txt.

Lesson for anyone repeating this check: absence of alerts in the first few seconds after an agent reconnects is not evidence the pipeline is broken. Wazuh only writes to alerts.log on a rule match, not on every raw event, and Windows Application-log events (scheduled tasks, service restarts, etc.) arrive on their own schedule rather than continuously the way Linux auth attempts do.

8. Detection Use Cases (Weeks 5–6) — all 4 PROVEN

All four use cases were validated with real evidence captured directly via SSH (jumping through the manager as a bastion to reach each agent's private IP), which is why the underlying instructions below are retained but every subsection opens with what was actually observed. The dashboard screenshots for all four use cases have since been captured and are embedded in Section 2.2.

8.1 Use case 1 — Centralised log ingestion

[DONE] PROVEN — PAM/sudo/sshd events flow continuously from both Linux agents with no additional setup required; confirmed directly in alerts.log.

Dashboard → Threat Hunting / Security Events, filter by agent.name, generate activity (e.g. a failed sudo) if the view is quiet, then expand one event and capture the full parsed document — raw log, matched rule, agent name. Repeat for the Windows agent to prove Windows Event Log data arrives too (also separately confirmed for the Windows agent specifically — Section 7.6).

8.2 Use case 2 — File Integrity Monitoring (FIM)

[DONE] PROVEN on agent-linux-1 — rules 554 (added), 550 (modified) and 553 (deleted) all fired with full hash-diff evidence in alerts.log; captured in week5/fim-config.md and week5/fim-explanation.md.

On agent-linux-1, add a monitored directory to the <syscheck> block of /var/ossec/etc/ossec.conf:

```
<directories check_all="yes" report_changes="yes" realtime="yes">/home/ubuntu/fim-
test</directories>
```

Restart the agent, then create, modify and delete a test file to trigger rules 554 (added), 550 (modified) and 553 (deleted). The modification alert (550) with the report_changes diff is the strongest single piece of evidence in the project — it shows the exact line an "attacker" added.

8.3 Use case 3 — Vulnerability detection

[DONE] PROVEN — 3,910 real findings already indexed against genuinely unpatched packages on the live agents; no need to deliberately install an old package to manufacture a CVE.

Enabled by default in Wazuh 4.14. Allow at least an hour after the manager starts for the CVE feed to populate (confirmed via "Feed update process completed" in the logs, including after the feed was wiped and re-downloaded following the disk-full incident). No staged package was needed — agent 001 already carried thousands of real unpatched-package findings. CVE list and expanded-detail screenshots captured; see Section 2.2.

8.4 Use case 4 — Authentication failure / brute force

[DONE] PROVEN — 12 failed SSH logins from agent-linux-2 against agent-linux-1's private IP fired rule 5710 x12 and the correlation rule 5712 (level 10), correctly attributing the source IP (10.0.11.208).

Since password authentication was disabled in Section 5.4, scripted failed SSH attempts against agent-linux-1 fail at the authentication stage — exactly the log line needed:

```
sudo apt-get install -y sshpass
for i in $(seq 1 12); do
  sshpass -p 'WrongPassword123' ssh -o StrictHostKeyChecking=no \
    -o PreferredAuthentications=password -o PubkeyAuthentication=no \
    baduser@10.0.1.21 2>/dev/null
done
```

Filter Security Events by rule.id (5710, 5716, 5712, 5551). Capture rule 5712 specifically — it is a correlation rule (fires after multiple failures from one source inside a time window), which demonstrates Wazuh's correlation engine rather than single-event matching. Individual failed logins are noise a SOC cannot alert on one at a time; rule 5710 fires per-attempt at level 5 (informational), while 5712 is a composite rule that only fires once several 5710/5716 events arrive from the same source inside the configured window, escalating to level 10 — the signal an analyst should actually be paged on. The source IP field is what would feed a blocklist or active-response rule in production.

8.5 Key rule IDs for the defence

Rule	Fires on	Level
554	File added to monitored directory	5
550	File modified in monitored directory	7
553	File deleted from monitored directory	7
5710	SSH login attempt, non-existent user	5
5716	SSH authentication failed	5
5712	SSHD brute force — correlated failures	10
5551	Related brute-force correlation	10
60122 / 60204	Windows RDP brute force	10

9. Risk Register

#	Risk	Likelihood	Impact	Mitigation
1	Public IP changes, lab appears broken	High	Low	Check IP at session start; update SG rule
2	t3.medium runs out of memory, indexer dies	Medium	High	4 GB swap added before install; monitor free -h
3	AWS costs exceed budget	Medium	Medium	Stop instances after every session; \$20 budget alert
4	Agent connects to public instead of private IP	Medium	High	Set WAZUH_MANAGER to private IP; verify with agent control -l
5	CVE feed not populated in time	Medium	Medium	Start manager early; allow 1+ hr for feed download
6	Lab breaks before evidence is captured	Medium	Critical	Capture every screenshot the moment it works
7	Locked out of SSH after hardening	Low	High	Keep an active session open while testing new config

#	Risk	Likelihood	Impact	Mitigation
8	Live demo fails on the day	Medium	High	Screenshot fallback ready in a second window
9	Windows agent fails to enrol via MSI	Low	Medium	Verify WazuhSvc running; check ossec.log
10	Credentials accidentally committed/screenshotted	Low	Critical	Redact at capture time; keep passwords outside evidence folder
11	REALISED 2026-09-22: manager disk fills up, crashing the indexer	—	High	Vulnerability-feed updater retried into a full disk; cleared by deleting the stale temp/feed cache and restarting services. Root cause not fully closed — check disk usage each session.
12	REALISED 2026-09-22/23: operator's ISP CGNAT breaks any SG CIDR restriction	—	Medium	Widened admin ports (22/443/3389) to 0.0.0.0/0 as a deliberate tradeoff after /32 and /21 both failed; SSH keys and dashboard/Windows login remain the real access control.
13	REALISED 2026-09-26: open admin ports (0.0.0.0/0) attract real internet brute-force scanning	—	Medium	Confirmed via dashboard evidence (471 real SSH failures + 981 real RDP failures in 24h, both correctly detected/correlated). Treated as a positive validation of the detection stack, but flagged for teardown/narrowing after grading (Section 2.3).

10. Eight-Week Plan and Checkpoints

Week	Focus	Status
1	Scope, AWS foundation planning, IAM/budget, SSH key, diagrams	DONE
2	VPC/SG/instances provisioned, hardened, inventory recorded	DONE (admin ports deliberately 0.0.0.0/0 — Section 5.2)
3	Wazuh manager installed (server+indexer+dashboard)	DONE — dashboard login screenshot captured (Section 2.2)
4	Linux + Windows agents enrolled	DONE — all 3 Active; dashboard screenshot captured
5	Use case 1 (log ingestion) + use case 2 (FIM) proven	DONE — SSH evidence + dashboard screenshots both captured
6	Use case 3 (vulnerability) + use case 4 (auth failure) proven	DONE — SSH evidence + dashboard screenshots both captured
7	Report, diagrams, troubleshooting log, improvement plan finalised	DONE — this document current, all evidence folded in
8	Live demonstration and hand-over; clean-up (terraform destroy)	IN PROGRESS — rehearsal script written, demonstration itself outstanding

Grading weights (from the project brief):

Area	Weight
Working SOC lab and completeness	30%
Cyber security execution and alert analysis	20%
Weekly milestones and working process	15%
Individual contribution	15%
Documentation and evidence	10%
Final demonstration and defence	10%

11. Evidence System

Evidence is tracked in SOCWatch-Evidence.zip using the folder structure below. Every screenshot is named weekN/NN-short-description.png and captioned with what it shows and why it matters. Redaction (passwords, tokens, account IDs, private key material) happens at capture time, never before submission.

```
SOCWatch-Evidence/  README.md  troubleshooting-log.md  instance-inventory.md  week1/ ..
week8/  diagrams/architecture-final.png  infra/ (Terraform)  scripts/ (session helpers)
report/SOCWatch-Final-Report.docx, SOCWatch-Presentation.pptx
```

12. Production Improvement Plan

Concrete gaps between this lab and a production-grade deployment, to be written up fully in the final report:

- Replace self-signed certificates with a proper CA-issued certificate.
- Put the dashboard behind a VPN or bastion host instead of a public IP allowlist.
- Split manager and indexer onto separate hosts; add a second manager node for high availability.
- Ship alerts to an external SIEM or S3 for retention beyond the indexer's window.
- Configure active response with carefully tuned thresholds, and document the risk of auto-blocking a legitimate admin.
- Add agent groups and centralised configuration so ossec.conf is not edited per host.
- Enable log archiving and integrity checks on the manager itself.
- Add MITRE ATT&CK mapping to detection rules for analyst triage context.

13. Quick Command Reference

Session start ritual

```
curl -s https://checkip.amazonaws.com aws ec2 start-instances --instance-ids i-... i-... --
profile socwatch
```

Manager health

```
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard
/var/ossec/bin/wazuh-control info
sudo /var/ossec/bin/agent_control -l
free -h
sudo tail -f /var/ossec/logs/ossec.log
```

Session end ritual

```
aws ec2 stop-instances --instance-ids i-... i-... --profile socwatch
```

14. Individual Contribution Statement

This project is completed individually. All AWS account configuration (IAM baseline, budget, security group design), infrastructure-as-code (Terraform), Wazuh installation, agent enrolment, detection use-case testing and reporting is designed, executed and defensible by the author, Nishen Madawa Abedeera (Student Number: REDACTED).